alpha fold

Rongyu Chen(A17708249)

2026-05-09

Generating your own structure predictions

Figure of 5 generated HIV_PR models

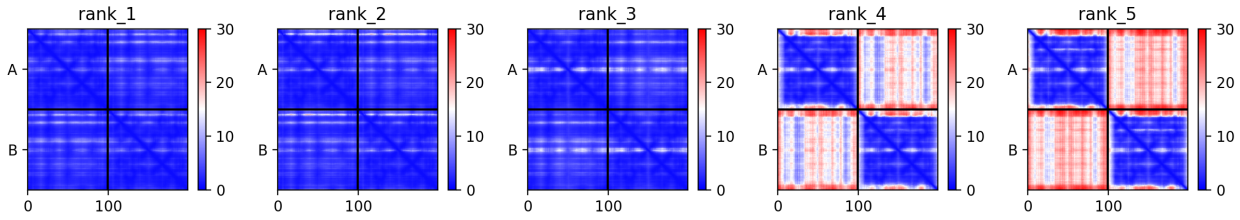


Figure 1: Figure of 5 generated HIV_PR models

Figure of sequence coverage

Figure of Predicted IDDT per position

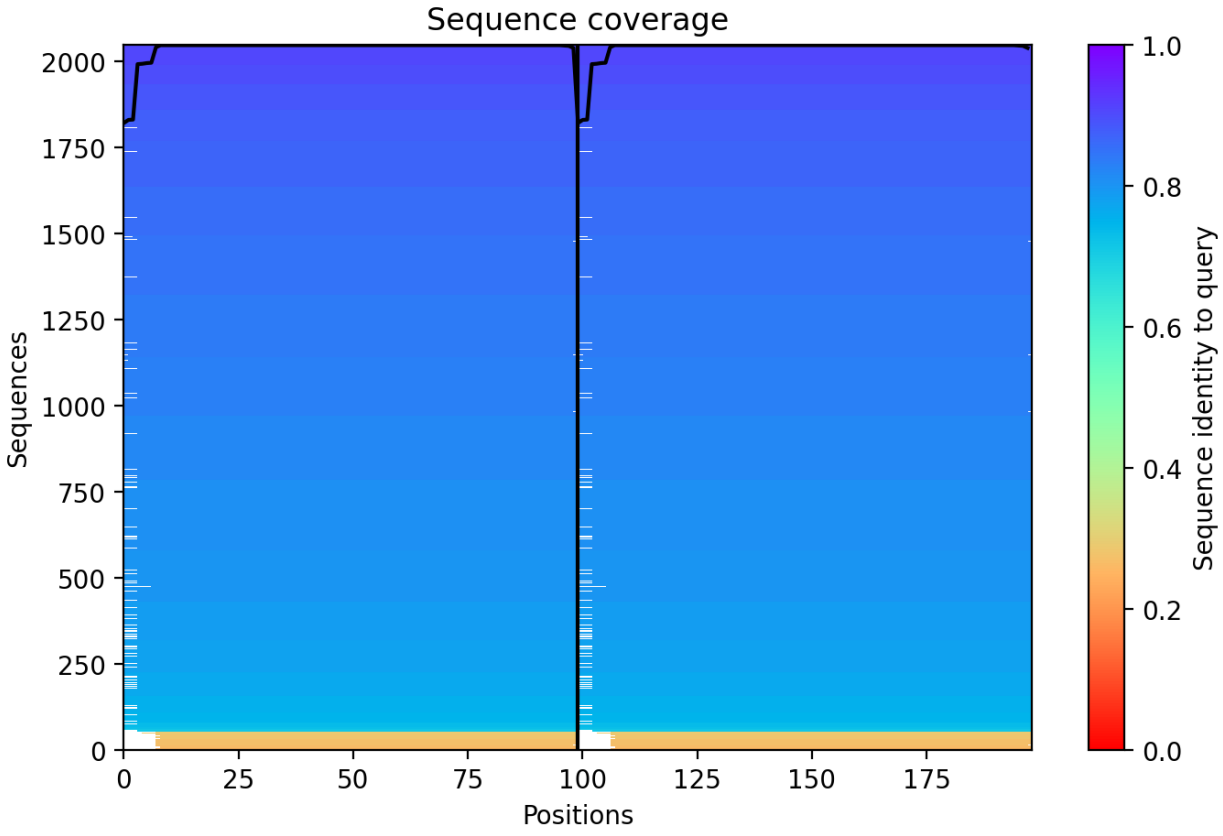
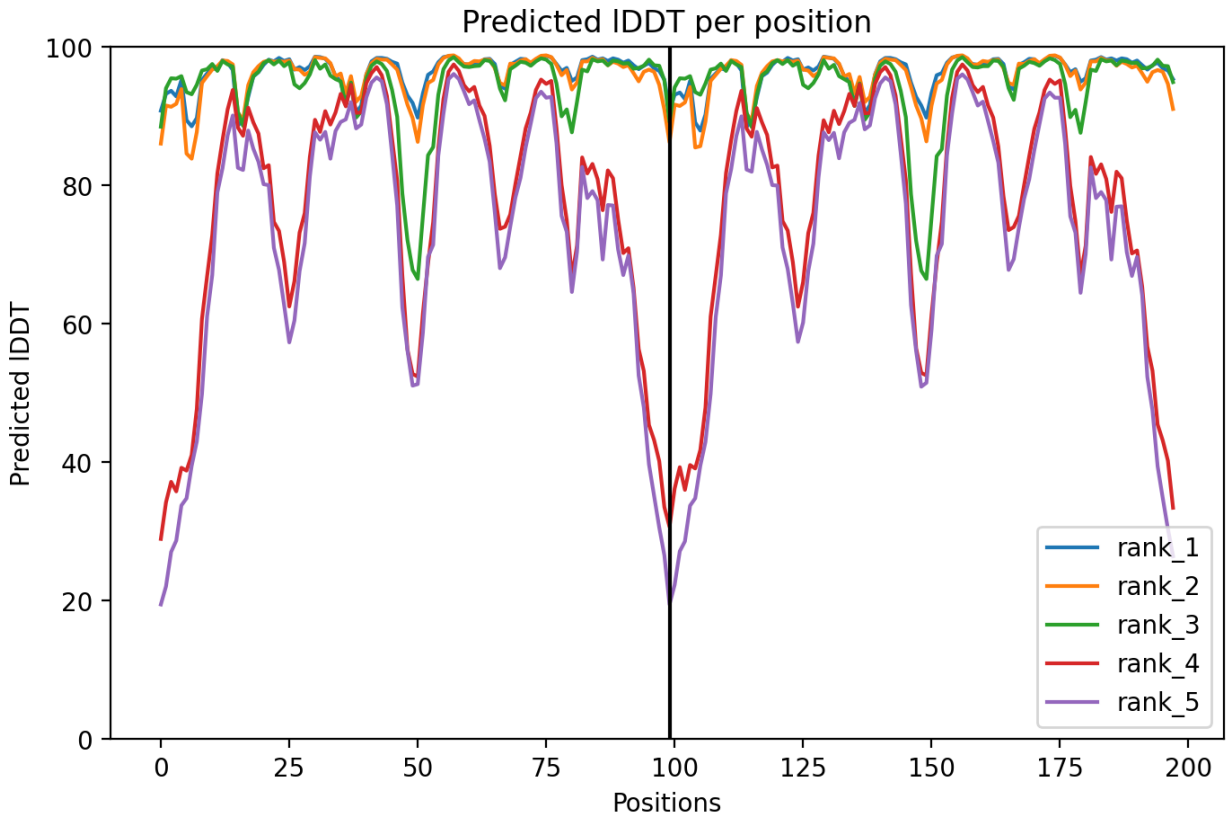


Figure 2: Figure of sequence coverage



Visualization of the models and their estimated reliability

Fitted and pLDDT colored Nras structure models.

Coloring the Residue property of Secondary Structure

Coloring the Residue property of Hydrophobicity

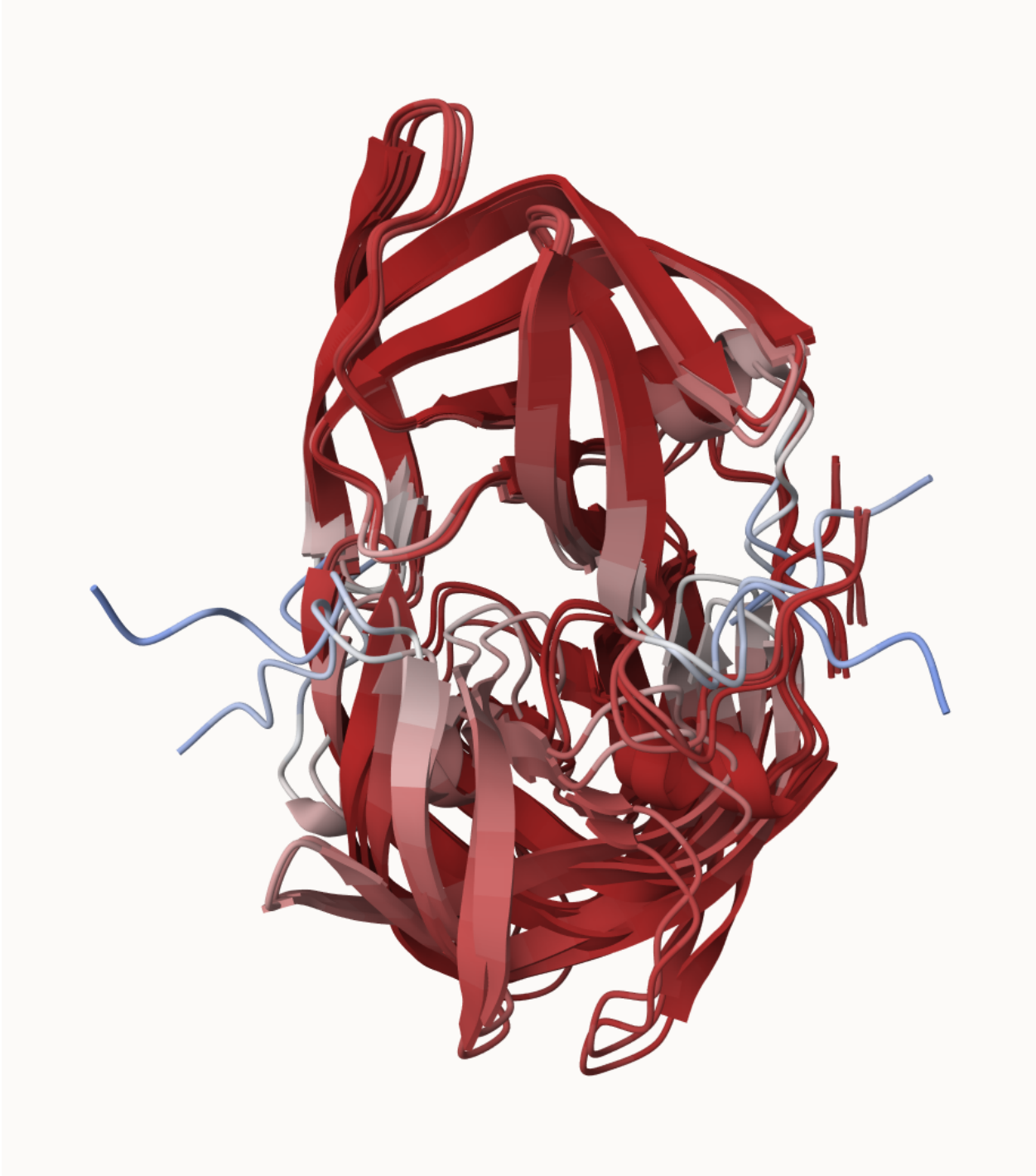


Figure 3: Fitted and pLDDT colored Nras structure models

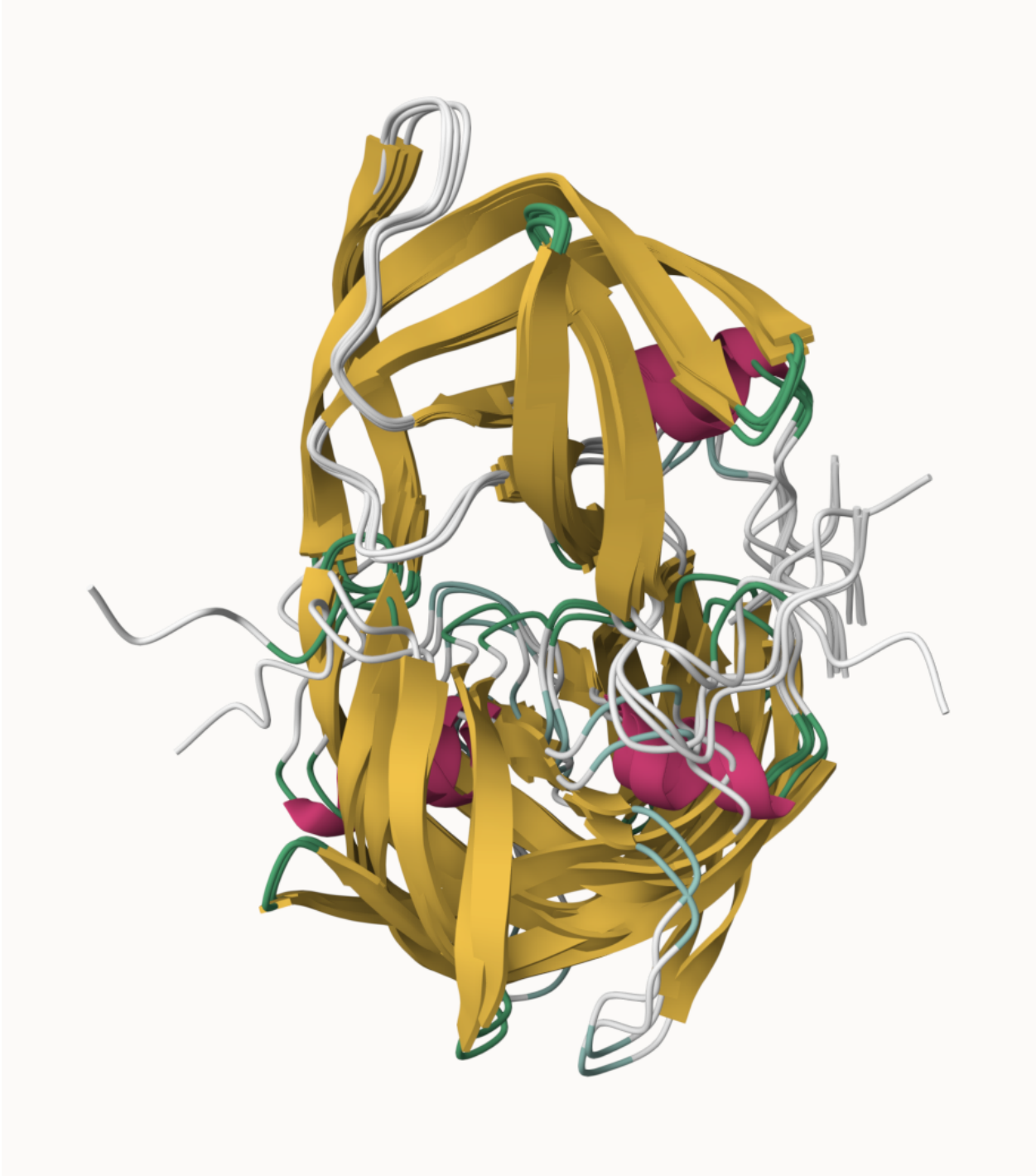
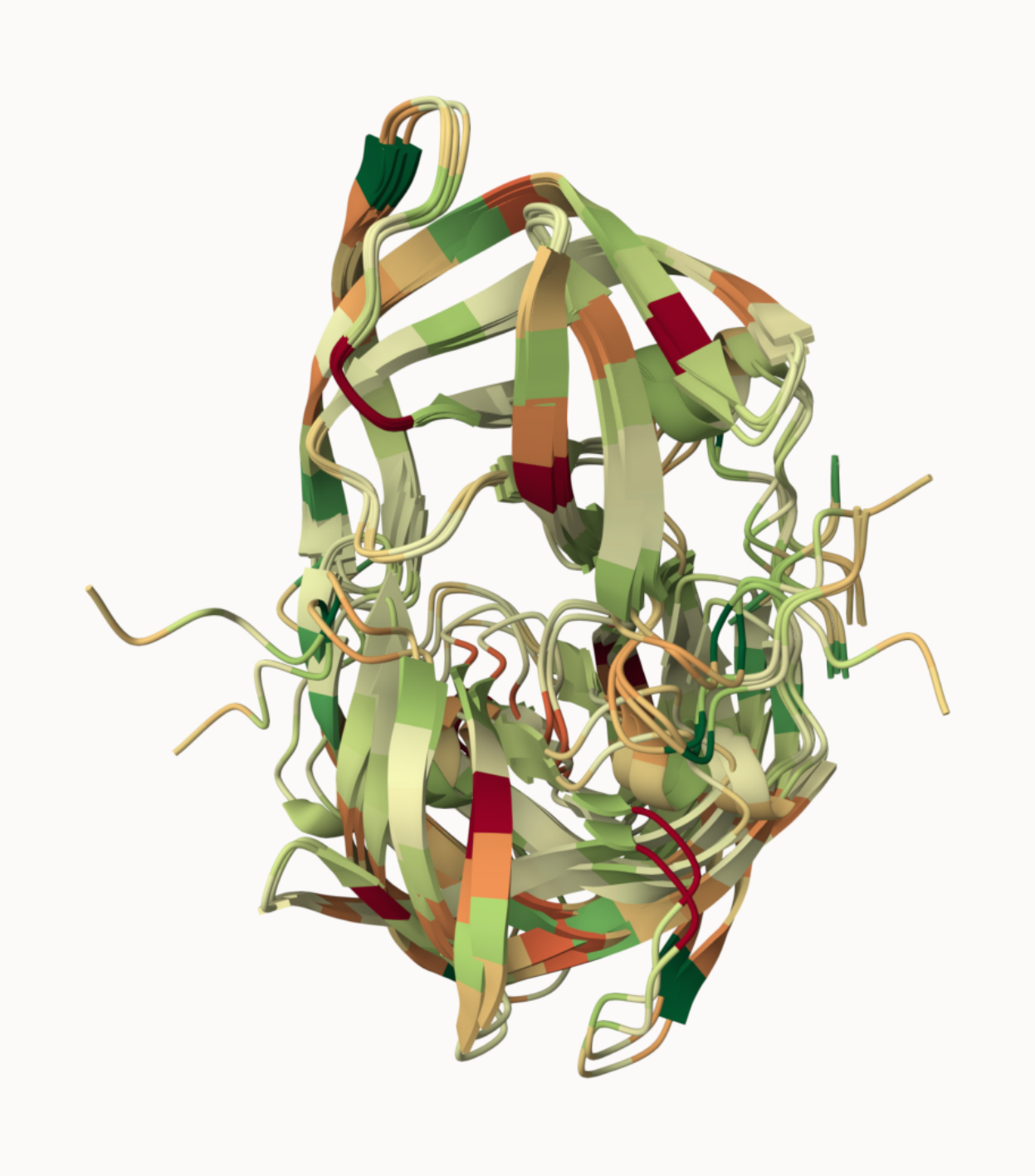


Figure 4: Secondary Structure



Coloring the Residue property of Accessible surface Area

Custom analysis of resulting models in R

Now we read the results of the more complicated HIV-Pr dimer AlphaFold2 models into R using `Bio3D package`.

To move our AlphaFold results directory into our RStudio project directory...

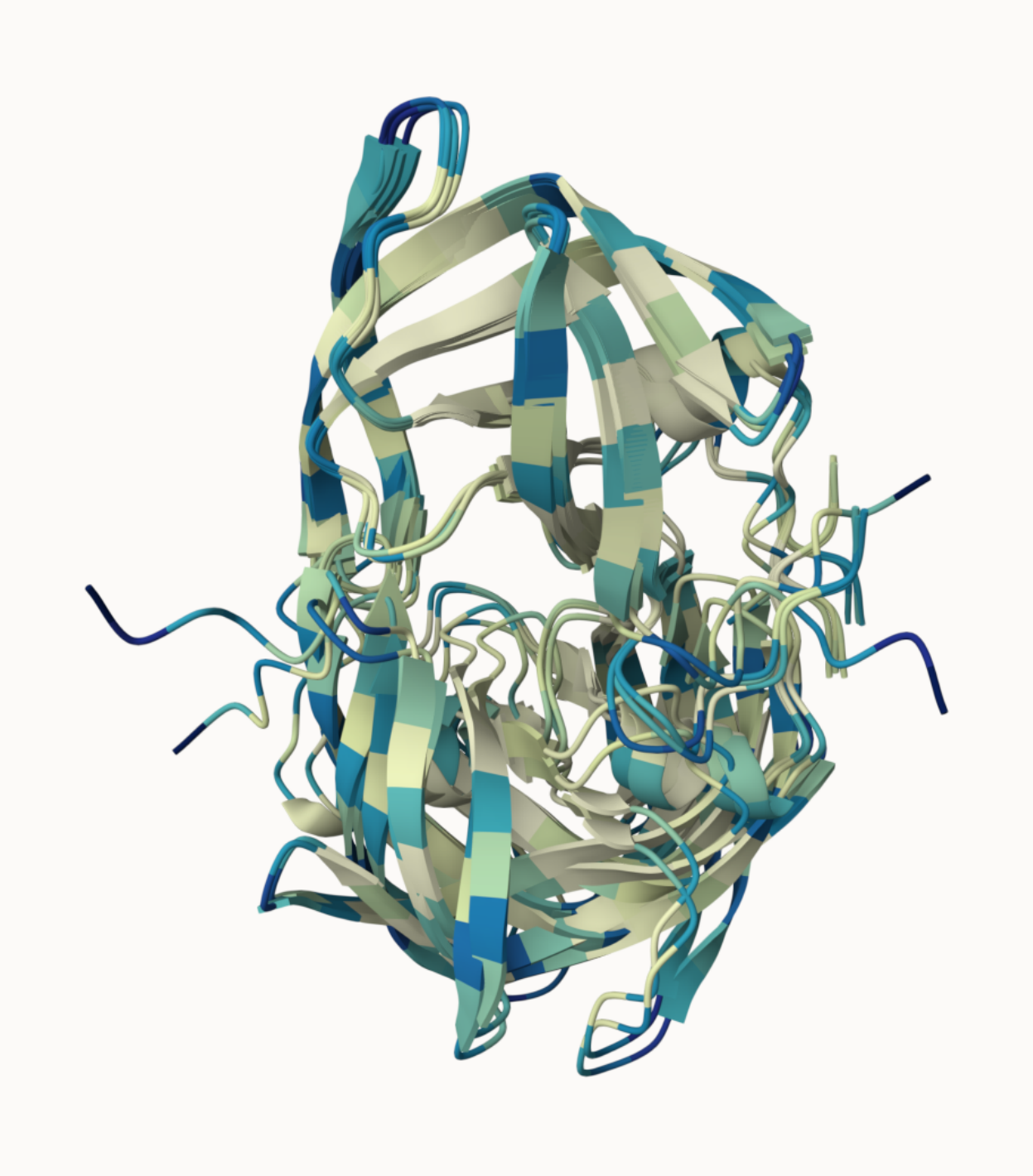


Figure 5: Accessible surface Area

```
results_dir <- "hivprdimer_23119/"

pdb_files <- list.files(path=results_dir,
                        pattern="*.pdb",
                        full.names = TRUE)
```

Print out our PDB file names

```
basename(pdb_files)
```

```
## [1] "test_23119_unrelaxed_rank_001_alphafold2_multimer_v3_model_4_seed_000.pdb"
## [2] "test_23119_unrelaxed_rank_002_alphafold2_multimer_v3_model_1_seed_000.pdb"
## [3] "test_23119_unrelaxed_rank_003_alphafold2_multimer_v3_model_5_seed_000.pdb"
## [4] "test_23119_unrelaxed_rank_004_alphafold2_multimer_v3_model_2_seed_000.pdb"
## [5] "test_23119_unrelaxed_rank_005_alphafold2_multimer_v3_model_3_seed_000.pdb"
```

```
library(bio3d)

pdbs <- pdbaln(pdb_files, fit=TRUE, exefile="msa")
```

```
## Reading PDB files:
## hivprdimer_23119/test_23119_unrelaxed_rank_001_alphafold2_multimer_v3_model_4_seed_000.pdb
## hivprdimer_23119/test_23119_unrelaxed_rank_002_alphafold2_multimer_v3_model_1_seed_000.pdb
## hivprdimer_23119/test_23119_unrelaxed_rank_003_alphafold2_multimer_v3_model_5_seed_000.pdb
## hivprdimer_23119/test_23119_unrelaxed_rank_004_alphafold2_multimer_v3_model_2_seed_000.pdb
## hivprdimer_23119/test_23119_unrelaxed_rank_005_alphafold2_multimer_v3_model_3_seed_000.pdb
## .....
##
## Extracting sequences
##
## pdb/seq: 1   name: hivprdimer_23119/test_23119_unrelaxed_rank_001_alphafold2_multimer_v3_model_4_seed
## pdb/seq: 2   name: hivprdimer_23119/test_23119_unrelaxed_rank_002_alphafold2_multimer_v3_model_1_seed
## pdb/seq: 3   name: hivprdimer_23119/test_23119_unrelaxed_rank_003_alphafold2_multimer_v3_model_5_seed
## pdb/seq: 4   name: hivprdimer_23119/test_23119_unrelaxed_rank_004_alphafold2_multimer_v3_model_2_seed
## pdb/seq: 5   name: hivprdimer_23119/test_23119_unrelaxed_rank_005_alphafold2_multimer_v3_model_3_seed
```

We can use the rmsd() function to calculate the RMSD between all pairs models.

```
rd <- rmsd(pdbs, fit=T)
```

```
## Warning in rmsd(pdbs, fit = T): No indices provided, using the 198 non NA positions
```

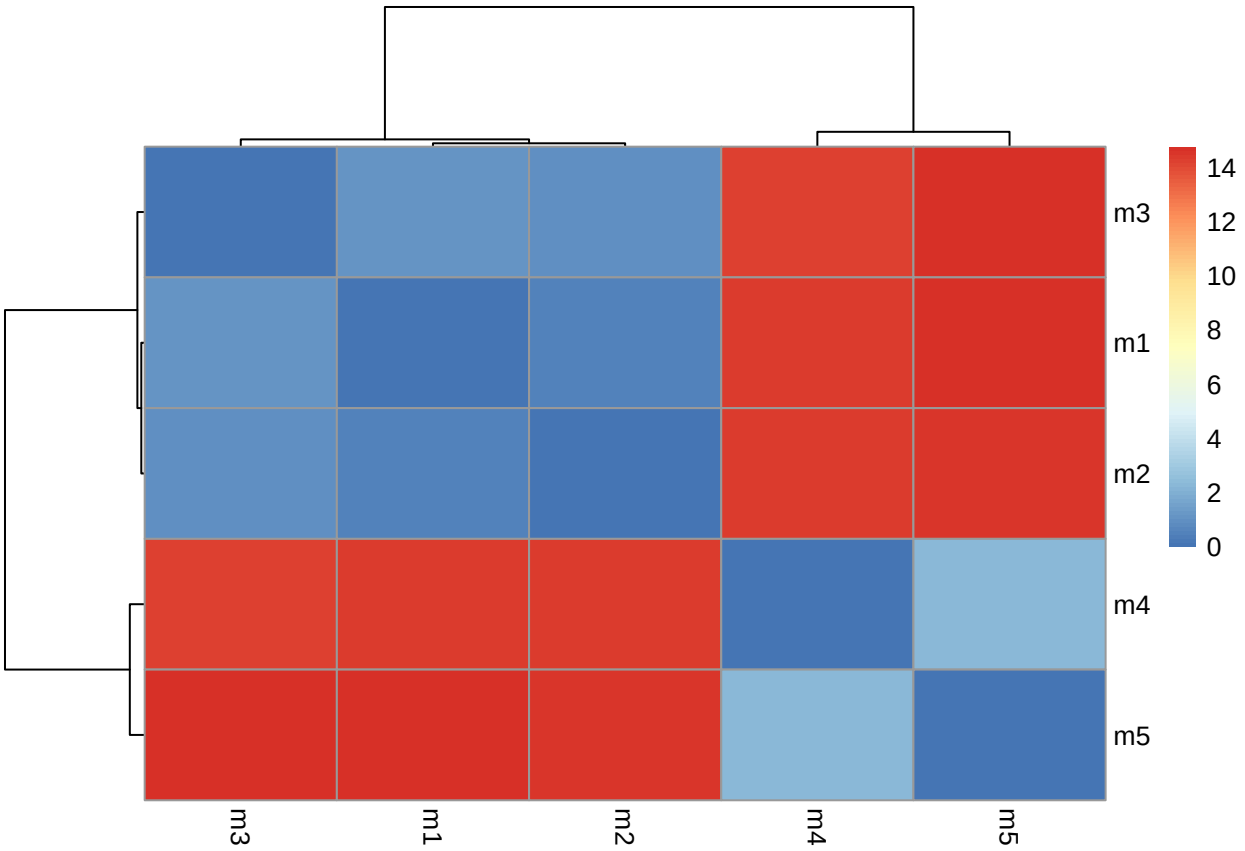
```
range(rd)
```

```
## [1]  0.000 14.754
```

Draw a heatmap of these RMSD matrix values

```
library(pheatmap)

colnames(rd) <- paste0("m",1:5)
rownames(rd) <- paste0("m",1:5)
pheatmap(rd)
```

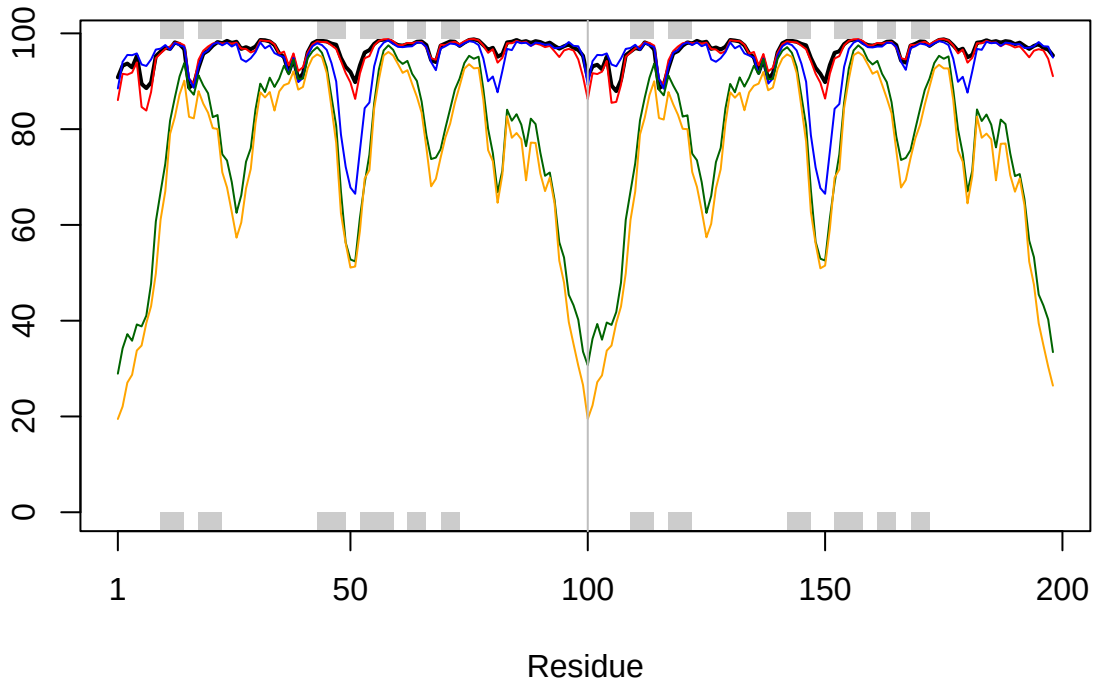


Models 1 and 2 are most similar to each other, while 4 and 5 resemble each other and are closer to 3 than to 1 or 2.

```
pdb <- read.pdb("1hsg")
```

```
##   Note: Accessing on-line PDB file
```

```
plotb3(pdbs$b[1,], typ="l", lwd=2, sse=pdb)
points(pdbs$b[2,], typ="l", col="red")
points(pdbs$b[3,], typ="l", col="blue")
points(pdbs$b[4,], typ="l", col="darkgreen")
points(pdbs$b[5,], typ="l", col="orange")
abline(v=100, col="gray")
```




```
core <- core.find(pdbs)
```

```
##  core size 197 of 198  vol = 9885.852
##  core size 196 of 198  vol = 6898.715
##  core size 195 of 198  vol = 1338.08
##  core size 194 of 198  vol = 1040.682
##  core size 193 of 198  vol = 951.866
##  core size 192 of 198  vol = 899.092
##  core size 191 of 198  vol = 834.742
##  core size 190 of 198  vol = 771.348
##  core size 189 of 198  vol = 733.074
##  core size 188 of 198  vol = 697.29
##  core size 187 of 198  vol = 659.751
##  core size 186 of 198  vol = 625.283
##  core size 185 of 198  vol = 589.55
##  core size 184 of 198  vol = 568.263
##  core size 183 of 198  vol = 545.024
##  core size 182 of 198  vol = 512.899
##  core size 181 of 198  vol = 490.733
##  core size 180 of 198  vol = 470.276
##  core size 179 of 198  vol = 450.74
##  core size 178 of 198  vol = 434.744
##  core size 177 of 198  vol = 420.346
##  core size 176 of 198  vol = 406.668
##  core size 175 of 198  vol = 393.343
##  core size 174 of 198  vol = 382.404
```

```
##  core size 173 of 198  vol = 372.868
##  core size 172 of 198  vol = 357.002
##  core size 171 of 198  vol = 346.576
##  core size 170 of 198  vol = 337.455
##  core size 169 of 198  vol = 326.668
##  core size 168 of 198  vol = 314.959
##  core size 167 of 198  vol = 304.136
##  core size 166 of 198  vol = 294.561
##  core size 165 of 198  vol = 285.658
##  core size 164 of 198  vol = 278.894
##  core size 163 of 198  vol = 266.775
##  core size 162 of 198  vol = 259.004
##  core size 161 of 198  vol = 247.732
##  core size 160 of 198  vol = 239.849
##  core size 159 of 198  vol = 234.972
##  core size 158 of 198  vol = 230.071
##  core size 157 of 198  vol = 221.994
##  core size 156 of 198  vol = 215.63
##  core size 155 of 198  vol = 206.802
##  core size 154 of 198  vol = 196.993
##  core size 153 of 198  vol = 188.548
##  core size 152 of 198  vol = 182.271
##  core size 151 of 198  vol = 176.962
##  core size 150 of 198  vol = 170.72
##  core size 149 of 198  vol = 166.128
##  core size 148 of 198  vol = 159.805
##  core size 147 of 198  vol = 153.775
##  core size 146 of 198  vol = 149.101
##  core size 145 of 198  vol = 143.666
##  core size 144 of 198  vol = 137.146
##  core size 143 of 198  vol = 132.525
##  core size 142 of 198  vol = 127.239
##  core size 141 of 198  vol = 121.581
##  core size 140 of 198  vol = 116.782
##  core size 139 of 198  vol = 112.577
##  core size 138 of 198  vol = 108.177
##  core size 137 of 198  vol = 105.14
##  core size 136 of 198  vol = 101.256
##  core size 135 of 198  vol = 97.381
##  core size 134 of 198  vol = 92.98
##  core size 133 of 198  vol = 88.19
##  core size 132 of 198  vol = 84.034
##  core size 131 of 198  vol = 81.904
##  core size 130 of 198  vol = 78.024
##  core size 129 of 198  vol = 75.278
##  core size 128 of 198  vol = 73.058
##  core size 127 of 198  vol = 70.701
##  core size 126 of 198  vol = 68.979
##  core size 125 of 198  vol = 66.709
##  core size 124 of 198  vol = 64.378
##  core size 123 of 198  vol = 61.147
##  core size 122 of 198  vol = 59.031
##  core size 121 of 198  vol = 56.627
##  core size 120 of 198  vol = 54.015
```

```
##  core size 119 of 198  vol = 51.805
##  core size 118 of 198  vol = 49.652
##  core size 117 of 198  vol = 48.193
##  core size 116 of 198  vol = 46.648
##  core size 115 of 198  vol = 44.752
##  core size 114 of 198  vol = 43.292
##  core size 113 of 198  vol = 41.093
##  core size 112 of 198  vol = 39.147
##  core size 111 of 198  vol = 36.471
##  core size 110 of 198  vol = 34.117
##  core size 109 of 198  vol = 31.47
##  core size 108 of 198  vol = 29.447
##  core size 107 of 198  vol = 27.325
##  core size 106 of 198  vol = 25.822
##  core size 105 of 198  vol = 24.15
##  core size 104 of 198  vol = 22.648
##  core size 103 of 198  vol = 21.069
##  core size 102 of 198  vol = 19.953
##  core size 101 of 198  vol = 18.3
##  core size 100 of 198  vol = 15.723
##  core size 99 of 198  vol = 14.841
##  core size 98 of 198  vol = 11.646
##  core size 97 of 198  vol = 9.434
##  core size 96 of 198  vol = 7.354
##  core size 95 of 198  vol = 6.179
##  core size 94 of 198  vol = 5.665
##  core size 93 of 198  vol = 4.705
##  core size 92 of 198  vol = 3.665
##  core size 91 of 198  vol = 2.77
##  core size 90 of 198  vol = 2.151
##  core size 89 of 198  vol = 1.715
##  core size 88 of 198  vol = 1.15
##  core size 87 of 198  vol = 0.874
##  core size 86 of 198  vol = 0.685
##  core size 85 of 198  vol = 0.528
##  core size 84 of 198  vol = 0.37
##  FINISHED: Min vol ( 0.5 ) reached
```

```
core.inds <- print(core, vol=0.5)
```

```
## # 85 positions (cumulative volume <= 0.5 Angstrom^3)
##   start end length
## 1     9  49     41
## 2    52  95     44
```

```
xyz <- pdbfit(pdbs, core.inds, outpath="corefit_structures")
```

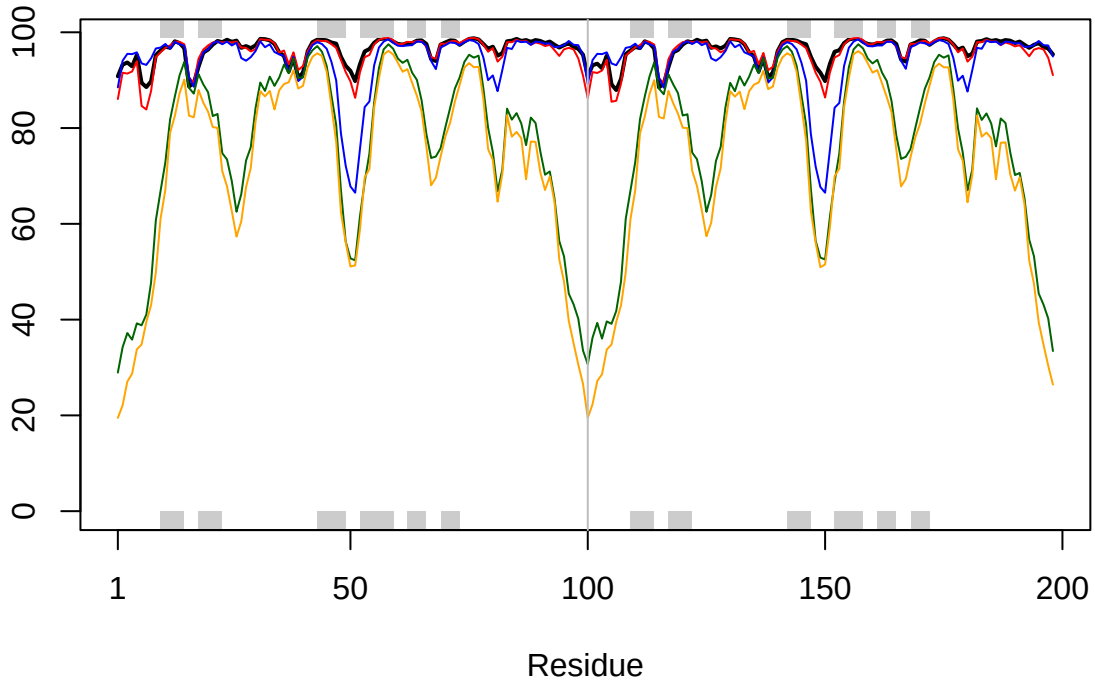
We can also plot the pLDDT values across all models

```
pdb <- read.pdb("1hsg")
```

```
##   Note: Accessing on-line PDB file
```

```
## Warning in get.pdb(file, path = tempdir(), verbose = FALSE):
## C:\Users\chenr\AppData\Local\Temp\RtmpeUBnOG/1hsg.pdb exists. Skipping download
```

```
plotb3(pdbs$b[1,], typ="l", lwd=2, sse=pdb)
points(pdbs$b[2,], typ="l", col="red")
points(pdbs$b[3,], typ="l", col="blue")
points(pdbs$b[4,], typ="l", col="darkgreen")
points(pdbs$b[5,], typ="l", col="orange")
abline(v=100, col="gray")
```



To enhance the alignment of our models, we can identify the most stable "rigid core" shared among them using the `core.find()` function:

```
core <- core.find(pdbs)
```

```
##  core size 197 of 198  vol = 9885.852
##  core size 196 of 198  vol = 6898.715
##  core size 195 of 198  vol = 1338.08
##  core size 194 of 198  vol = 1040.682
##  core size 193 of 198  vol = 951.866
##  core size 192 of 198  vol = 899.092
##  core size 191 of 198  vol = 834.742
##  core size 190 of 198  vol = 771.348
##  core size 189 of 198  vol = 733.074
##  core size 188 of 198  vol = 697.29
##  core size 187 of 198  vol = 659.751
##  core size 186 of 198  vol = 625.283
##  core size 185 of 198  vol = 589.55
##  core size 184 of 198  vol = 568.263
##  core size 183 of 198  vol = 545.024
```

```
##  core size 182 of 198  vol = 512.899
##  core size 181 of 198  vol = 490.733
##  core size 180 of 198  vol = 470.276
##  core size 179 of 198  vol = 450.74
##  core size 178 of 198  vol = 434.744
##  core size 177 of 198  vol = 420.346
##  core size 176 of 198  vol = 406.668
##  core size 175 of 198  vol = 393.343
##  core size 174 of 198  vol = 382.404
##  core size 173 of 198  vol = 372.868
##  core size 172 of 198  vol = 357.002
##  core size 171 of 198  vol = 346.576
##  core size 170 of 198  vol = 337.455
##  core size 169 of 198  vol = 326.668
##  core size 168 of 198  vol = 314.959
##  core size 167 of 198  vol = 304.136
##  core size 166 of 198  vol = 294.561
##  core size 165 of 198  vol = 285.658
##  core size 164 of 198  vol = 278.894
##  core size 163 of 198  vol = 266.775
##  core size 162 of 198  vol = 259.004
##  core size 161 of 198  vol = 247.732
##  core size 160 of 198  vol = 239.849
##  core size 159 of 198  vol = 234.972
##  core size 158 of 198  vol = 230.071
##  core size 157 of 198  vol = 221.994
##  core size 156 of 198  vol = 215.63
##  core size 155 of 198  vol = 206.802
##  core size 154 of 198  vol = 196.993
##  core size 153 of 198  vol = 188.548
##  core size 152 of 198  vol = 182.271
##  core size 151 of 198  vol = 176.962
##  core size 150 of 198  vol = 170.72
##  core size 149 of 198  vol = 166.128
##  core size 148 of 198  vol = 159.805
##  core size 147 of 198  vol = 153.775
##  core size 146 of 198  vol = 149.101
##  core size 145 of 198  vol = 143.666
##  core size 144 of 198  vol = 137.146
##  core size 143 of 198  vol = 132.525
##  core size 142 of 198  vol = 127.239
##  core size 141 of 198  vol = 121.581
##  core size 140 of 198  vol = 116.782
##  core size 139 of 198  vol = 112.577
##  core size 138 of 198  vol = 108.177
##  core size 137 of 198  vol = 105.14
##  core size 136 of 198  vol = 101.256
##  core size 135 of 198  vol = 97.381
##  core size 134 of 198  vol = 92.98
##  core size 133 of 198  vol = 88.19
##  core size 132 of 198  vol = 84.034
##  core size 131 of 198  vol = 81.904
##  core size 130 of 198  vol = 78.024
##  core size 129 of 198  vol = 75.278
```

```
##  core size 128 of 198  vol = 73.058
##  core size 127 of 198  vol = 70.701
##  core size 126 of 198  vol = 68.979
##  core size 125 of 198  vol = 66.709
##  core size 124 of 198  vol = 64.378
##  core size 123 of 198  vol = 61.147
##  core size 122 of 198  vol = 59.031
##  core size 121 of 198  vol = 56.627
##  core size 120 of 198  vol = 54.015
##  core size 119 of 198  vol = 51.805
##  core size 118 of 198  vol = 49.652
##  core size 117 of 198  vol = 48.193
##  core size 116 of 198  vol = 46.648
##  core size 115 of 198  vol = 44.752
##  core size 114 of 198  vol = 43.292
##  core size 113 of 198  vol = 41.093
##  core size 112 of 198  vol = 39.147
##  core size 111 of 198  vol = 36.471
##  core size 110 of 198  vol = 34.117
##  core size 109 of 198  vol = 31.47
##  core size 108 of 198  vol = 29.447
##  core size 107 of 198  vol = 27.325
##  core size 106 of 198  vol = 25.822
##  core size 105 of 198  vol = 24.15
##  core size 104 of 198  vol = 22.648
##  core size 103 of 198  vol = 21.069
##  core size 102 of 198  vol = 19.953
##  core size 101 of 198  vol = 18.3
##  core size 100 of 198  vol = 15.723
##  core size 99 of 198  vol = 14.841
##  core size 98 of 198  vol = 11.646
##  core size 97 of 198  vol = 9.434
##  core size 96 of 198  vol = 7.354
##  core size 95 of 198  vol = 6.179
##  core size 94 of 198  vol = 5.665
##  core size 93 of 198  vol = 4.705
##  core size 92 of 198  vol = 3.665
##  core size 91 of 198  vol = 2.77
##  core size 90 of 198  vol = 2.151
##  core size 89 of 198  vol = 1.715
##  core size 88 of 198  vol = 1.15
##  core size 87 of 198  vol = 0.874
##  core size 86 of 198  vol = 0.685
##  core size 85 of 198  vol = 0.528
##  core size 84 of 198  vol = 0.37
##  FINISHED: Min vol ( 0.5 ) reached
```

The resulting core atom positions then serve as a reference for a refined superposition, after which we can save the fitted structures to `corefit_structures`:

```
core.inds <- print(core, vol=0.5)
```

```
## # 85 positions (cumulative volume <= 0.5 Angstrom^3)
##   start end length
## 1     9  49     41
```

```
## 2    52  95     44
```

```
xyz <- pdbfit(pdbs, core.inds, outpath="corefit_structures")
```

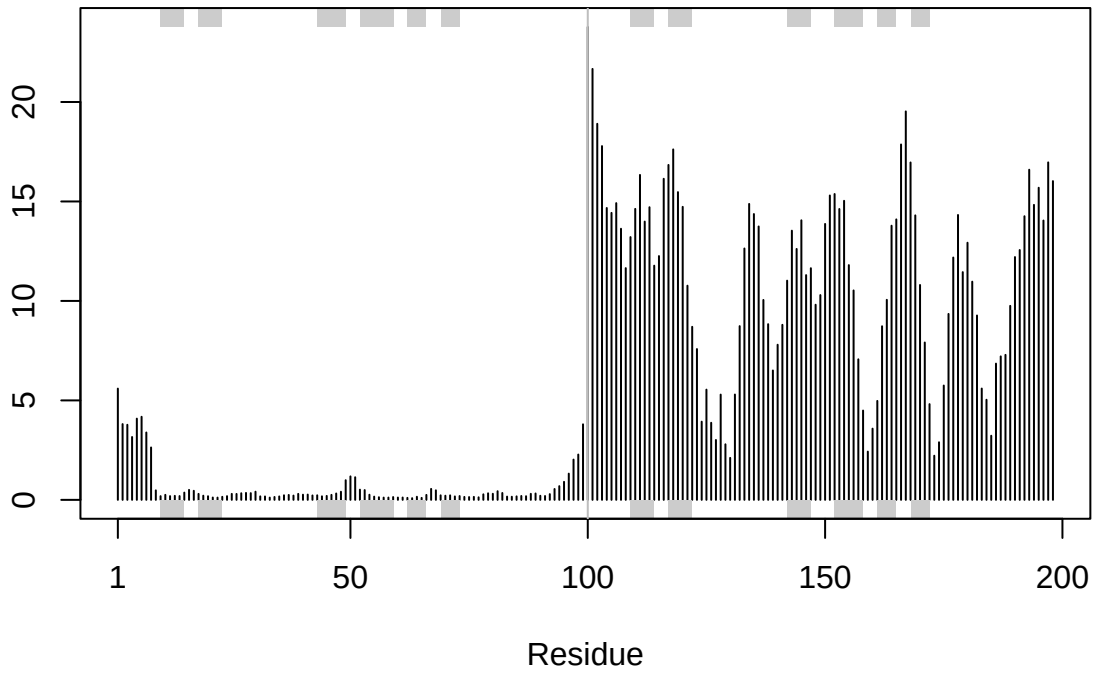
The resulting superposed coordinates are written to a new director called `corefit_structures/`. We can now open these in Mol* and color by the **Atom Property** of **Uncertainty/Disorder**

/Users/chenr/Downloads/M1_CONSERV.PDB-2.png

Examine the RMSF between positions of the structure.

```
rf <- rmsf(xyz)

plotb3(rf, sse=pdb)
abline(v=100, col="gray", ylab="RMSF")
```



Predicted Alignment Error for domains

AlphaFold also provides a *Predicted Aligned Error (PAE)* output, stored in per-model JSON files. PAE measures the expected positional error between residues, independent of the 3D structure.

In R, we can read these files with jsonlite

```
library(jsonlite)

pae_files <- list.files(path=results_dir,
                        pattern=".*model.*\\.json",
                        full.names = TRUE)
```

for example purposes lets read the 1st and 5th files (you can read the others and make similar plots).

Figure 6: Figure of m1_conserv Uncertainty

```
pae1 <- jsonlite::read_json(pae_files[1], simplifyVector = TRUE)
pae5 <- jsonlite::read_json(pae_files[5], simplifyVector = TRUE)
attributes(pae1)
```

```
## $names
## [1] "plddt"   "max_pae" "pae"     "ptm"     "iptm"
```

```
head(pae1$plddt)
```

```
## [1] 90.81 93.25 93.69 92.88 95.25 89.44
```

The maximum PAE values are useful for ranking models. Here we can see that model 5 is much worse than model 1. The lower the PAE score the better. How about the other models, what are thir max PAE scores?

```
pae1$max_pae
```

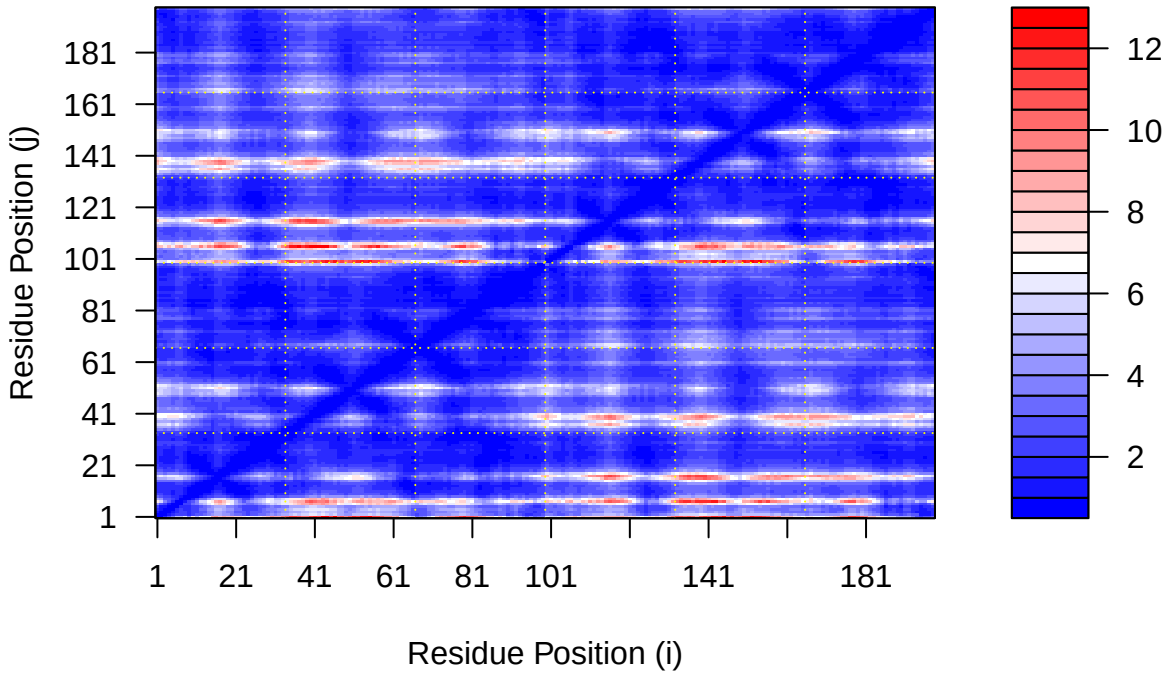
```
## [1] 12.84375
```

```
pae5$max_pae
```

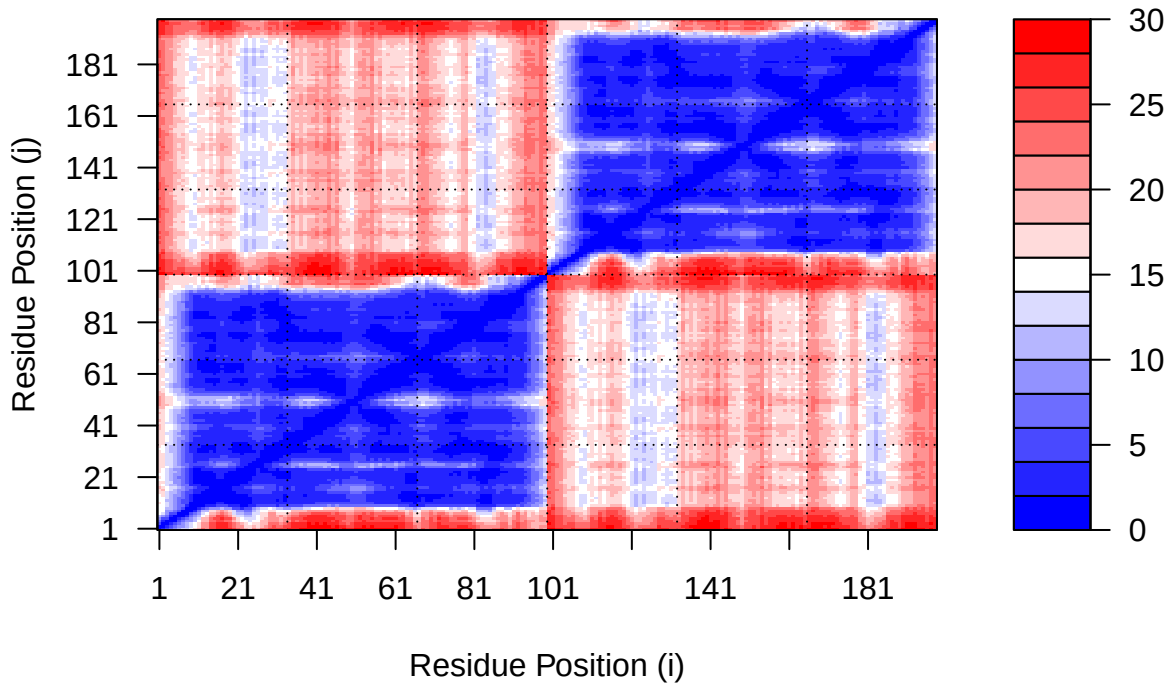
```
## [1] 29.59375
```

We can plot the N by N (where N is the number of residues) PAE scores with ggplot or with functions from the Bio3D package:

```
plot.dmat(pae1$pae,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)")
```

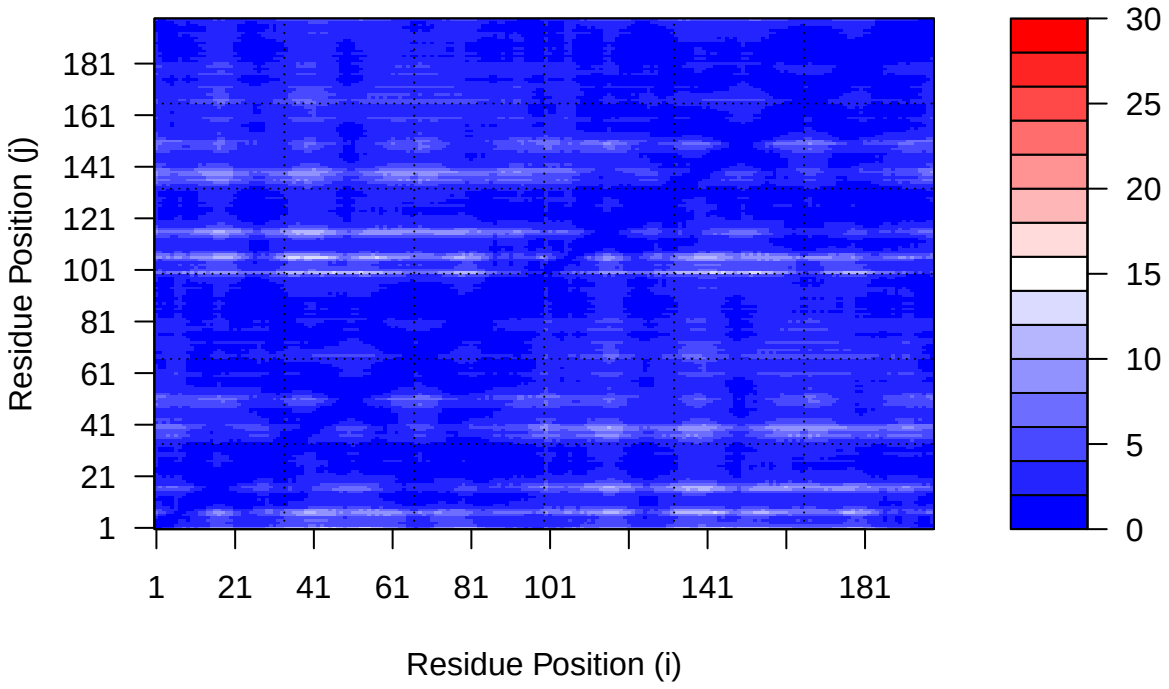


```
plot.dmat(pae5$pae,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)",
          grid.col = "black",
          zlim=c(0,30))
```



We should really plot all of these using the same z range. Here is the model 1 plot again but this time using the same data range as the plot for model 5:

```
plot.dmat(pae1$pae,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)",
          grid.col = "black",
          zlim=c(0,30))
```



Residue conservation from alignment file

```
aln_file <- list.files(path=results_dir,
                       pattern=".a3m$",
                        full.names = TRUE)
aln_file
```

```
## [1] "hivprdimer_23119/test_23119.a3m"
```

```
aln <- read.fasta(aln_file[1], to.upper = TRUE)
```

```
## [1] " ** Duplicated sequence id's: 101 **"
## [2] " ** Duplicated sequence id's: 101 **"
```

How many sequences are in this alignment

```
dim(aln$ali)
```

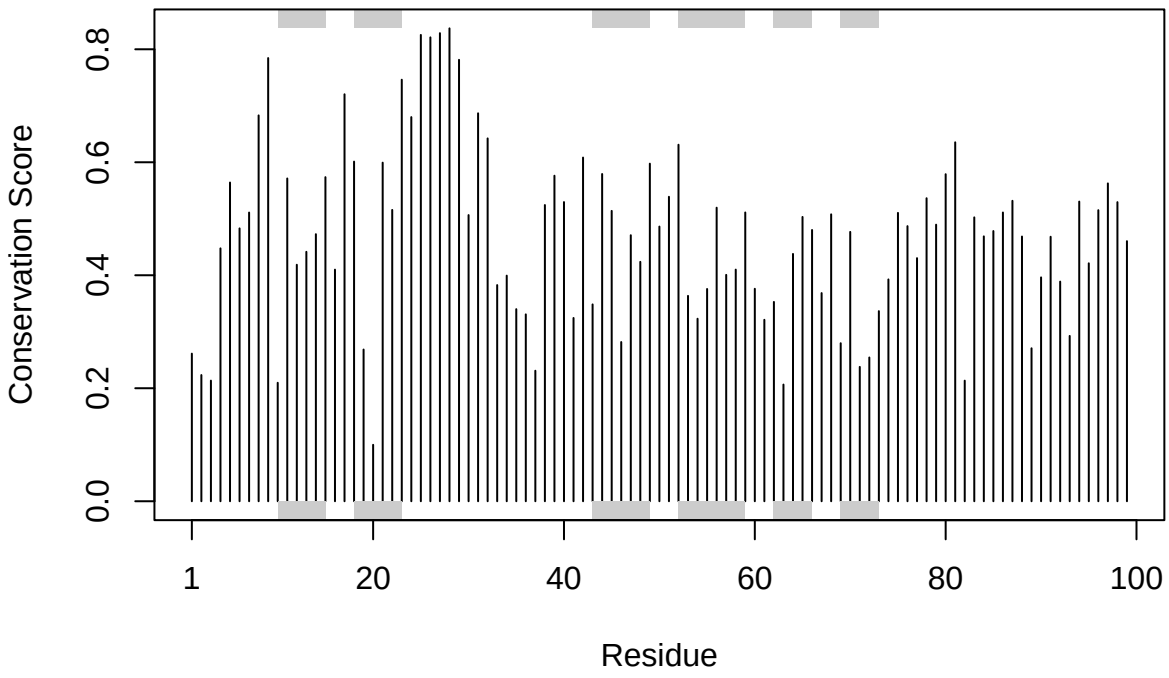
```
## [1] 5397  132
```

The alignment contains 5,397 sequences and 132 positions.

We can score residue conservation in the alignment with the `conserv()` function.

```
sim <- conserv(aln)

plotb3(sim[1:99], sse=trim.pdb(pdb, chain="A"),
       ylab="Conservation Score")
```



Note the conserved Active Site residues D25, T26, G27, A28. These positions will stand out if we generate a consensus sequence with a high cutoff value:

```
con <- consensus(aln, cutoff = 0.9)
con$seq
```

```
##   [1] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
##  [19] "-" "-" "-" "-" "-" "-" "D" "T" "G" "A" "-" "-" "-" "-" "-" "-" "-" "-" "-"
##  [37] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
##  [55] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
##  [73] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
##  [91] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
## [109] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
## [127] "-" "-" "-" "-" "-" "-"
```

For a final visualization of these functionally important sites we can map this conservation score to the Occupancy column of a PDB file for viewing in molecular viewer programs such as Mol*, PyMol, VMD, chimera etc.

```
m1.pdb <- read.pdb(pdb_files[1])
occ <- vec2resno(c(sim[1:99], sim[1:99]), m1.pdb$atom$resno)
write.pdb(m1.pdb, o=occ, file="m1_conserv.pdb")
```

```
getwd()
```

```
## [1] "C:/Users/chenr/OneDrive/Desktop/alpha fold"
```

```
list.files()
```

```
##  [1] "Accessible_Surface_Area.png"
##  [2] "aln.fa"
##  [3] "alpha-fold.Rmd"
##  [4] "alpha-fold_files"
##  [5] "alpha fold.Rmd"
##  [6] "alpha fold.Rproj"
##  [7] "corefit_structures"
##  [8] "Fitted_and_pLDDT_colored_Nras_structure models.png"
##  [9] "hivprdimer_23119"
## [10] "HIVPRDimer_23119_coverage.png"
## [11] "HIVPRDimer_23119_pae.png"
## [12] "HIVPRDimer_23119_plddt.png"
## [13] "Hydrophobicity.png"
## [14] "m1_conserv.pdb"
## [15] "M1_CONSERV.PDB-2.png"
## [16] "M1_CONSERV.PDB.png"
## [17] "Secondary_structure.png"
```

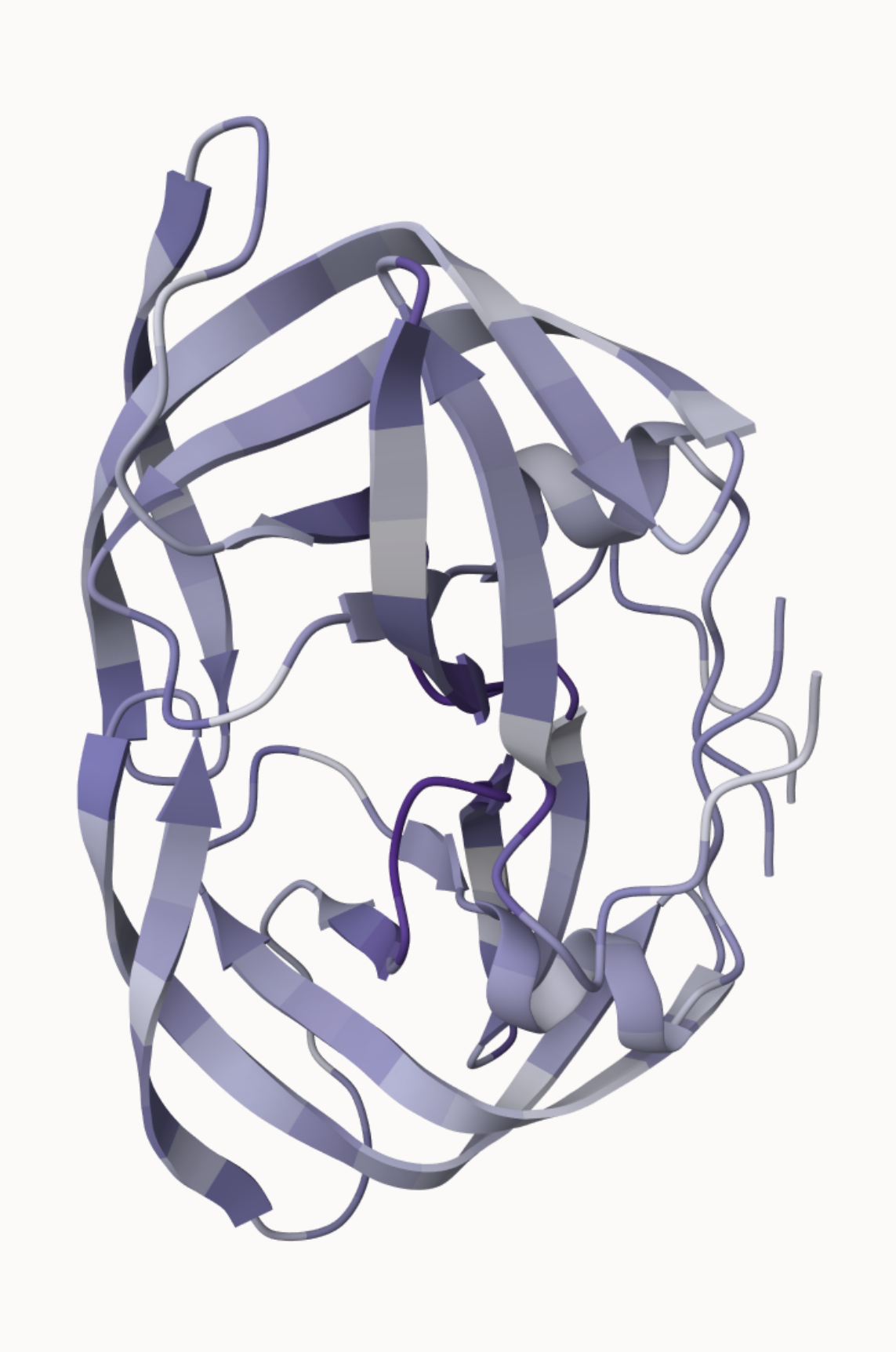


Figure 7: Figure of m1_conserv Occupancy